COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 15, 2026	**DATE SUBMITTED:**	October 20, 2026
NAME:	Dizor, Elijah Gresola, Aryanna Keith	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13.1 **Parallelism and Multithreading**

1. PROCEDURE OUTPUT

ILO A & B:

Part 1: Simple One-Threaded Example

INPUT

SINGLE THREAD > main.cpp > ...

```cpp
#include <iostream>
#include <thread>

void print(int n, const std::string &str){
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main (){
    std::thread t1(print, 10, "T.I.P");
    t1.join();

return 0;
}
```

OUTPUT

GRESOLA

```
Printing integer: 10
Printing string: T.I.P
```

DIZOR

```
printing integer:10
printing string: T.I.P
```

ANALYSIS

In this program, we observed how the thread executes the print() function by displaying the integer 10 first, followed by the string "T.I.P.". The output follows the order of the statements inside the function. We also tried removing t1.join() to understand its purpose. Without it, the main thread could reach the end of the program while t1 was still running, causing the program to terminate because the thread was still joinable. This showed us that join() is important because it makes the main thread wait for the assigned task to finish before the program ends. We also considered replacing it with t2.join(), but since t2 was not declared in the code, the program would not compile. Through this, we learned that join() must refer to an existing thread and helps ensure that threads are properly managed.

Part 2: Multi-Threaded Example

INPUT

MULTITHREAD > main.cpp > ...

```cpp
#include <iostream>
#include <string>
#include <thread>
#include <vector>

void print(int n, const std::string &str){
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main (){
    std::vector<std::string> s = {
        "T.I.P",
        "Competent",
        "Computer",
        "Engineers"
    };
    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++){
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads){
        th.join();
    }
    return 0;
}
```

OUTPUT	
GRESOLA 1 : Competent 2 : Computer 3 : Engineers 0 : T.I.P	DIZOR 2 : Competent 1 : T.I.P 3 : Computer 4 : Engineers

ANALYSIS

In this program, we observed that the four threads could display their messages in different orders every time we ran the code. This is because each thread runs independently, and the operating system decides which thread executes first. We also tried removing the join() loop, and the program ended up terminating because the threads were still joinable, meaning they had been started but had not yet been joined or detached. When a joinable thread object is destroyed, the program calls std::terminate(). The th.join() statement is important because it makes the main thread wait for each thread to finish. Even if we change the order of the join() statements, the messages can still appear in a different order because join() does not control which thread runs first. Through this activity, we learned that multithreading allows different tasks to run concurrently, but their execution order is not always the same.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

QUESTIONS	
1. Write a definition of multithreading and its advantages/disadvantages.	
DIZOR	Multi - threading is an execution that lets one program divide itself into several concurrent pathways. It operates within the same memory space, these threads run concurrently or switch or dynamically execute different parts of an application simultaneously. Some of its advantages are being able to move heavy calculations or downloads to background threads, where the application interface stays responsive, and never freezes up during long tasks. It also has smart resource sharing to create and switch between threads to reduce time, and better CPU performance to enable web servers to handle thousands of user requests or data streams. However, some of its disadvantages are the risk of data corruption, when sharing the same memory threads, it can be accidentally overwritten. It is also unpredictable debugging, where the threads are executed in a random order determined by the operating system
GRESOLA	Multithreading is a technique in programming where a program uses multiple threads to handle different tasks during overlapping periods. To understand it simply, imagine cooking a meal. While the rice is cooking, you can prepare the ingredients and cook another dish instead of waiting for one task to finish before starting the next. In programming, multithreading works similarly because different threads can handle separate tasks within the same program. One advantage is that it can make a program more responsive and use the CPU more efficiently. However, it can also be difficult to manage because threads may run in different orders, and problems can occur when they access or change the same data at the same time.
2. Rationalize the use of multithreading by providing at least 3 use-cases.	
DIZOR	Multi - threading helps to improve performance and resources of all applications by running multiple tasks at the same time. Some examples are the use of google chrome, or any other web browsers, it utilizes multiple threads and processes to keep your browning seamless. Applications like youtube, which relies on heavy multithreading to decode and encode video data on your devices that are uploaded on its servers. Especially facebook, an application that aggregates data from several sources to construct a single homepage feed
GRESOLA	Multithreading is useful when a program needs to handle several tasks without making the entire program wait for just one task to finish. For example, in a video game, one thread can handle the game's logic while another processes sound, helping the game remain responsive. In a music player, one thread can play music while another responds to the user's actions, such as pausing the song or skipping to the next track. Another example is downloading files,

	where a program can manage the download while still allowing the user to browse the interface or cancel the process. These examples show how multithreading helps programs manage different tasks more efficiently and improve the user's experience.

3. Differentiate between parallelism and concurrency.

DIZOR	Concurrency occurs when two or more tasks can be run, and completed in overlapping time periods, as they do not run at the exact time speeds. It switches back and forth between tasks faster. Parallelism occurs when two or more tasks run at the exact same time but it requires hardware support, multiple CPUs where each core independently executes a separate task at the same time
GRESOLA	Concurrency is when a program handles multiple tasks by switching between them, even if they are not done at the exact same time. For example, when I am doing schoolwork, I can answer my worksheet first and then switch to reading a lesson material while managing both tasks. On the other hand, parallelism is when multiple tasks are done at the same time. For example, in a group project, one member can prepare the presentation while another writes the documentation simultaneously. In short, concurrency focuses on managing multiple tasks, while parallelism focuses on executing multiple tasks at the same time.

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

GRESOLA

INPUT

PART B > main.cpp > ...

```cpp
#include <iostream>
#include <thread>
#include <mutex>

int globalValue = 0;
std::mutex mtx;

void add(int value) {
    std::lock_guard<std::mutex> lock(mtx);
    globalValue += value;
}

void display() {
    std::lock_guard<std::mutex> lock(mtx);
    std::cout << "Global value: "
              << globalValue << std::endl;
}

int main() {
    std::thread T1(add, 5);
    std::thread T2(add, 15);
    std::thread T3(add, 25);

    display();

    T1.join();
    display();

    T2.join();
    display();

    T3.join();

    return 0;
}
```

OUTPUT

```
Global value: 0
Global value: 45
Global value: 45
```

ANALYSIS

In this program what is being done is that three threads are created to add 5, 15, and 25 to the global variable. At first, I was confused as to why the second and third displays showed the same value of 45. After researching, I learned that the three threads can run at the same time, so they may finish adding their values before the program reaches the second display. The join() function only waits for a specific thread to finish and does not stop the other threads from running. This explains why the value can already be 45 after T1.join() and remain 45 after T2.join(). Through this activity, I learned that the execution of threads can happen in different orders, which affects the displayed values.

DIZOR

INPUT

```
#include <iostream>
#include <thread>
#include <chrono>
#include <iomanip>
#include "counter.h"

void log_status(const std::string& message, int current_value) {
    std::cout << std::left << std::setw(30) << message
              << "Global Counter Value: " << current_value << std::endl;
}

int main() {
    int val1 = 0, val2 = 0, val3 = 0;

    std::cout << "Enter an integer for Thread 1: ";
    std::cin >> val1;
    std::cout << "Enter an integer for Thread 2: ";
    std::cin >> val2;
    std::cout << "Enter an integer for Thread 3: ";
    std::cin >> val3;

    auto start_time = std::chrono::high_resolution_clock::now();

    std::thread T1(add, val1);
    std::thread T2(add, val2);
    std::thread T3(add, val3);

    log_status("Initial Display", get_global_counter());

```

```
30      T1.join();
31      log_status("T1.join()", get_global_counter());
32
33      T2.join();
34      log_status("T2.join()", get_global_counter());
35
36      T3.join();
37      log_status("T3.join()", get_global_counter());
38
39      auto end_time = std::chrono::high_resolution_clock::now();
40      std::chrono::duration<double, std::milli> total_duration = end_time - start_time;
41
42      return 0;
43  }
44
```

Header

```
1   #ifndef COUNTER_H
2   #define COUNTER_H
3
4   #include <thread>
5   #include <chrono>
6
7   inline int& get_global_counter() {
8       static int counter = 0;
9       return counter;
10  }
11
12  inline void add(int value) {
13      std::this_thread::sleep_for(std::chrono::milliseconds(100));
14      get_global_counter() += value;
15  }
16
17  #endif
18
```

OUTPUT

```
Enter an integer for Thread 1: 15
Enter an integer for Thread 2: 7
Enter an integer for Thread 3: 31

Initial Display                 Global Counter Value: 0
T1.join()                       Global Counter Value: 53
T2.join()                       Global Counter Value: 53
T3.join()                       Global Counter Value: 53

--------------------------------
Process exited after 2.898 seconds with return value 0
Press any key to continue . . .
```

ANALYSIS

It allocates memory for free local variables, then it blocks sequentially at three separate input boundaries. The program halts until you give 3 integer inputs and after receiving the values, it inserts a newline break to visually isolate the input from its multi-threaded computation. After the inputs are registered, it creates a high resolution time anchor using ‘chrono’ and ‘resolution ‘clock’

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

GRESOLA

INPUT

PART C > main.cpp > ...

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>
#include <iomanip>
#include <functional>

using Clock = std::chrono::steady_clock;

// single threaded - linear search
int linearSearch(const std::vector<int>& arr, int target){
    for (int i = 0; i < static_cast<int>(arr.size()); i++){
        if (arr[i] == target){
            return i;
        }
    }

    return -1;
}

void searchPart(const std::vector<int>& arr, int target,
                int start, int end, int& result){
    for (int i = start; i < end; i++){
        if (arr[i] == target){
            result = i;
            return;
        }
    }
}

// multithreaded - linear search
int parallelLinearSearch(const std::vector<int>& arr, int target){
    int mid = static_cast<int>(arr.size()) / 2;
    int result1 = -1;
    int result2 = -1;

    std::thread T1(searchPart, std::cref(arr), target,
                   0, mid, std::ref(result1));

    std::thread T2(searchPart, std::cref(arr), target,
                   mid, static_cast<int>(arr.size()),
                   std::ref(result2));
```

```
    T1.join();
    T2.join();

    if (result1 != -1){
        return result1;
    }
    return result2;
}

int main(){
    const int SIZE = 10000;
    const int TARGET = 10000;
    std::vector<int> arr(SIZE);
    for (int i = 0; i < SIZE; i++){
        arr[i] = i + 1;
    }

    // single threaded search
    auto start1 = Clock::now();
    int result1 = linearSearch(arr, TARGET);
    auto end1 = Clock::now();
    double time1 =
        std::chrono::duration<double, std::micro>(
            end1 - start1).count();

    // multithreaded search
    auto start2 = Clock::now();
    int result2 = parallelLinearSearch(arr, TARGET);
    auto end2 = Clock::now();
    double time2 =
        std::chrono::duration<double, std::micro>(
            end2 - start2).count();

    std::cout << std::fixed << std::setprecision(3);
    std::cout << "Performance Comparison" << std::endl;;
    std::cout << "Array size: " << SIZE << std::endl;
    std::cout << "Target: " << TARGET << std::endl;
    std::cout << "----------------------------" << std::endl;

    std::cout << "Single-threaded result: " << result1 << std::endl;
    std::cout << "Single-threaded time: "
              << time1 << " microseconds\n\n";

    std::cout << "Multithreaded result: " << result2 << std::endl;
    std::cout << "Multithreaded time: "
              << time2 << " microseconds" << std::endl;

    return 0;
}
```

OUTPUT	
SMALLER INPUT SIZE	LARGER INPUT SIZE
10000: Performance Comparison Array size: 10000 Target: 10000 ---------- Single-threaded result: 9999 Single-threaded time: 71.200 microseconds Multithreaded result: 9999 Multithreaded time: 10714.800 microseconds	1000000: Performance Comparison Array size: 1000000 Target: 1000000 ---------- Single-threaded result: 999999 Single-threaded time: 4739.100 microseconds Multithreaded result: 999999 Multithreaded time: 2931.200 microseconds

ANALYSIS

In this activity, I used Linear Search to compare the performance of single-threaded and multithreaded execution. Based on my observations, the single-threaded version was faster when using smaller input sizes because it searches the elements directly without the additional overhead of creating and managing threads. However, when I increased the input size to 1,000,000 elements, the multithreaded version became more efficient in my test because the workload was divided between two threads, allowing both parts of the array to be searched simultaneously. This activity helped me understand that multithreading can improve performance when handling larger amounts of data, while single-threaded execution can be more efficient for smaller tasks.

DIZOR

INPUT

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>
#include <random>
#include <algorithm>
#include <climits>

void find_partial_min(const std::vector<int>& arr, int start, int end, int& output_min) {
    int local_min = INT_MAX;
    for (int i = start; i < end; ++i) {
        if (arr[i] < local_min) {
            local_min = arr[i];
        }
    }
    output_min = local_min;
}

void evaluate_performance(int workload_size) {
    std::vector<int> data_pool(workload_size);
    std::random_device rd;
    std::mt19937 generator(rd());
    std::uniform_int_distribution<> distribution(-500000, 500000);

    for (int i = 0; i < workload_size; ++i) {
        data_pool[i] = distribution(generator);
    }

    std::cout << "Elements checked: " << workload_size << std::endl;
    std::cout << "Search boundary: " << INT_MAX << std::endl;
    std::cout << "\n" << std::endl;

    auto seq_start = std::chrono::high_resolution_clock::now();

    int sequential_minimum = INT_MAX;
    for (int i = 0; i < workload_size; ++i) {
        if (data_pool[i] < sequential_minimum) {
            sequential_minimum = data_pool[i];
        }
    }

    auto seq_end = std::chrono::high_resolution_clock::now();
    auto seq_duration = std::chrono::duration_cast<std::chrono::nanoseconds>(seq_end - seq_start).count() / 1000.0;

    std::cout << "Sequential result found: " << sequential_minimum << std::endl;
    std::cout << "Sequential processing duration: " << seq_duration << " microseconds" << std::endl;
    std::cout << std::endl;

    auto parallel_start = std::chrono::high_resolution_clock::now();
```

```
    const int THREAD_POOL_SIZE = 4;
    std::thread worker_pool[THREAD_POOL_SIZE];
    int intermediate_mins[THREAD_POOL_SIZE] = { INT_MAX, INT_MAX, INT_MAX, INT_MAX };

    int segment_width = workload_size / THREAD_POOL_SIZE;

    for (int i = 0; i < THREAD_POOL_SIZE; ++i) {
        int left_boundary = i * segment_width;
        int right_boundary = (i == THREAD_POOL_SIZE - 1) ? workload_size : (left_boundary + segment_width);

        worker_pool[i] = std::thread(find_partial_min,
                                     std::ref(data_pool),
                                     left_boundary,
                                     right_boundary,
                                     std::ref(intermediate_mins[i]));
    }

    int final_parallel_minimum = INT_MAX;
    for (int i = 0; i < THREAD_POOL_SIZE; ++i) {
        worker_pool[i].join();
        if (intermediate_mins[i] < final_parallel_minimum) {
            final_parallel_minimum = intermediate_mins[i];
        }
    }

    auto parallel_end = std::chrono::high_resolution_clock::now();
    auto parallel_duration = std::chrono::duration_cast<std::chrono::nanoseconds>(parallel_end - parallel_start).count() / 1000.0;

    std::cout << "Parallel pool result: " << final_parallel_minimum << std::endl;
    std::cout << "Parallel pool duration: " << parallel_duration << " microseconds" << std::endl;
    std::cout << "\n" << std::endl;
}

int main() {
    evaluate_performance(5000);
    evaluate_performance(2000000);
    return 0;
}
```

OUTPUT	
SMALL INPUT	LARGE INPUT
5000	20000
Elements checked: 5000 Search boundary: 2147483647 Sequential result found: -499953 Sequential processing duration: 0 microseconds Parallel pool result: -499953 Parallel pool duration: 0 microseconds	Elements checked: 2000000 Search boundary: 2147483647 Sequential result found: -500000 Sequential processing duration: 6968.1 microseconds Parallel pool result: -500000 Parallel pool duration: 2994.4 microseconds

ANALYSIS

It starts by creating a large array and uses a random number generator to fill the entire array with positive and negative numbers. In the single thread, it starts a timer and a single loop checks every single element in the array from start to finish, keeping track of the lowest value it finds and then the stops the timer and calculates the

execution duration. At multi thread search, it launches four separate threads simultaneously and each thread is assigned to search only its specific quarter of the array, running simultaneously on different CPU cores to avoid data conflicts, each thread saves the lowest number it found in its assigned section.

CONCLUSION

DIZOR	Learning the use of multi threading and parallelism, made me understand that physical limits prevent single processor clock speeds from accelerating further and modern computation power scales by adding more. As modern applications demand fluid interfaces, muli - threading helps to prevent slow mathematical workloads from blocking executions paths which preserves application and responsiveness and user experience
GRESOLA	Through this activity, I learned that a program's runtime depends on the algorithm used, the operation being performed, and the capabilities of the device. Different algorithms and approaches can produce different performance results depending on the problem and input size. Therefore, it is important to choose the most suitable algorithm for a specific problem to optimize execution time and improve program efficiency.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction, code assistance, and explanation of concepts.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: ChatGPT was used to improve grammar, clarify programming concepts, and assist with parts of the code that we did not fully understand. However, it did not replace our original thinking, analysis, authorship, or completion of the required work.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.